

- GAME DESIGN DOCUMENT -

BLACKOUT PROTOCOL

MARIA HUHNSTOCK

Table of contents

1. Pitch....1
2. Design Pillars....1
3. Core Loop....1
4. Mechanics....2
    4.1 Power System....2
    4.2 Structures....2
    4.3 Light & Visibility....2
    4.4 Enemies (2 Types)....3
    4.5 Waves....3
    4.6 Scrap & Rebuilding....3
    4.7 Rank & Evaluation....4
    4.8 Playtesting & Balancing....4
    4.9 Audio....4
5. UI / UX....4
6. Artstyle....5
7. Technical Structure....6
8. Scope....8
9. Possible Extensions....8

Blackout Protocol - Game Design Document

GENRE: Top-Down Strategy / Tower Defense **ENGINE:** Unity 6.6 (URP), C#

1. Pitch

An outpost at night, a generator with exactly 10 units of power, and three waves of enemies crawling in from the dark. You don’t build anything – everything is already standing. Your job is to decide, second by second, what gets power: the lamps that reveal enemies, the turret that shoots them, or the drone that repairs what they damage. There is never enough for everything at once.

2. Design Pillars

- **Power is the currency** - every decision is about the same 10 units. Lamps, turret and drone together need 11, so something always stays off.
- **Light defines the battlefield** - the turret only hits enemies standing in light. Switching off a lamp saves power but opens a blind spot.
- **Readable in the dark** - clear silhouettes, color-coded structures and an always-visible power display make the situation understandable at a glance, even when most of the map is black.
- **Recover, don’t restart** - losing a lamp is a setback, not the end. Ruins can be rebuilt between waves.

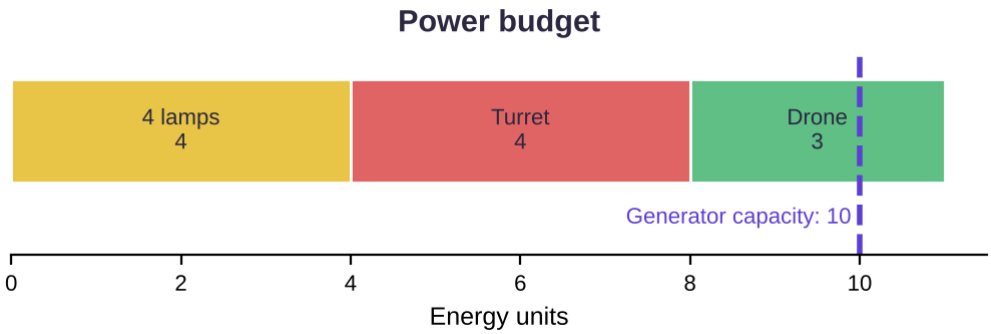
3. Core Loop

1. The mission starts with a preparation phase: lamps are on, turret and drone are off.
2. You plan the power distribution. Space starts the wave early.
3. Enemies enter from the left and right edges of the map and look for a target.
4. During the wave you reroute power with mouse clicks: light up the side under attack, switch on the turret, or power the drone for repairs.
5. Destroyed enemies drop scrap.
6. Once all enemies are defeated, the next preparation phase begins – destroyed structures can now be rebuilt with scrap.
7. After the third wave the mission is won and rated with a rank. If the generator is destroyed → blackout, the mission is lost.

4. Mechanics

4.1 Power System

- The generator supplies 10 units of power.
- Clicking a structure switches it on or off. Switching on only works if enough free power is left – otherwise the request is denied and the power display flashes red.
- If the capacity ever drops below the current usage, the most recently powered consumers switch off first until usage fits again (load shedding).



Lamps, turret and drone together need 11 units – one more than the generator delivers.

4.2 Structures

STRUCTURE	POWER	HP	FUNCTION
Lamp (×4)	1	4	Lights a circle around itself. Only enemies inside a lit area can be targeted.
Turret	4	8	Fires a laser at the nearest lit enemy within 5 cells, two shots per second.
Repair drone	3	5	Flies to the most damaged structure within 6 cells and repairs it (2 HP per second).
Generator	–	20	Supplies the power and emits a faint light of its own. If it is destroyed, the mission is lost.

All structure values live in StructureData ScriptableObjects (TurretData and RepairData extend them), so balancing happens in the Inspector instead of in code.

4.3 Light & Visibility

- There is no ambient light: everything outside a powered lamp is pitch black.
- The visible light radius of a lamp is also its gameplay radius. Before targeting, the turret checks whether a position is lit. Only 80% of the visible radius counts, so enemies at the faint edge of the light are not targeted.
- The generator has its own faint light, so the last line of defense always remains defendable.

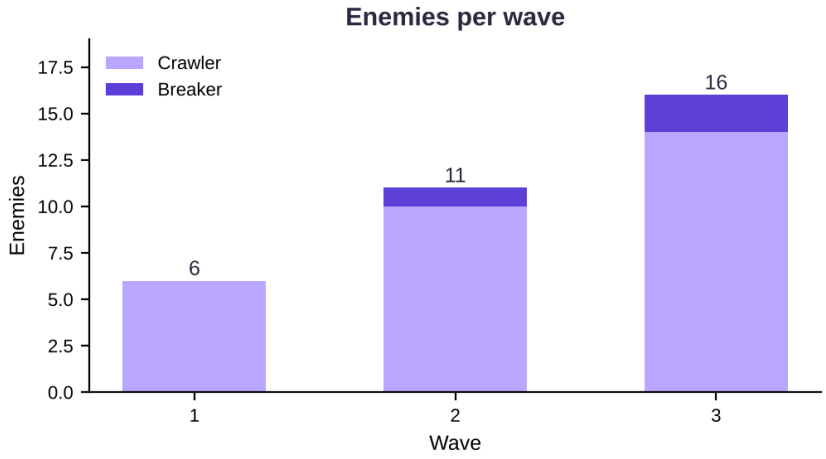
4.4 Enemies (2 Types)

TYPE	BEHAVIOR	THREAT
Crawler	Small and fast (3 HP). Attacks the nearest structure – often a lamp. Comes in swarms.	Medium
Breaker	Slow and tough (12 HP). Ignores everything else and heads straight for the generator, dealing heavy damage (3 per hit).	High

Both enemy types run on the same EnemyController. They differ only in their EnemyData and TargetingStrategy assets (nearest structure vs. generator first, with a fallback if the generator is gone), so a new enemy type mostly needs new data, not new code.

4.5 Waves

- Three waves, each defined in a WaveData ScriptableObject as parallel spawn groups (enemy type, count, start delay, interval).
- 30 seconds of preparation before the first wave, 20 seconds between waves. Space skips the countdown.
- A wave ends when every enemy of that wave has been defeated.



Wave 2 introduces the first Breaker, wave 3 combines the largest Crawler swarm with two Breakers.

4.6 Scrap & Rebuilding

- Defeated enemies drop scrap: Crawler 1, Breaker 5.
- Destroyed structures (except the generator) release their power and leave a ruin behind.
- During the preparation phase, ruins show their name and rebuild cost – green if affordable, red if not. A click rebuilds the structure.
- Rebuilt structures return at full health but switched off, so they have to be powered again.

STRUCTURE	REBUILD COST (SCRAP)
Lamp	3
Turret	8
Repair drone	6

4.7 Rank & Evaluation

- The results screen shows time, enemies defeated, structures lost and the average power usage during the waves.
- On victory, the rank depends on structures lost: S = none, A = up to 2, B = up to 5, C = more. Rebuilt structures still count as lost.
- On defeat, no rank is awarded.

4.8 Playtesting & Balancing

The values weren’t set once and left alone – they were tested and adjusted in several rounds. Three changes came directly out of playtesting: a dim ambient light kept enemies readable outside the lamps, so switching a lamp off barely felt like a cost – the ambient light was removed entirely. The generator first sat in the dark, which let a single Breaker destroy it unopposed – it now emits a faint light of its own. And once a lamp was destroyed, its area stayed dark for the rest of the mission, so runs were effectively lost minutes before the generator fell – ruins and rebuilding with scrap now make mistakes recoverable. The power costs are deliberately set so that running everything at once needs exactly one unit more than the generator supplies.

4.9 Audio

- **Two music loops** - a calm one for the preparation phase and a driving one during waves, both in the same key and looping seamlessly.
- **Crossfade** - the music fades smoothly between preparation and wave.
- **Stingers** - short pieces for victory and defeat.
- **Ducking** - music is turned down while the game is paused.

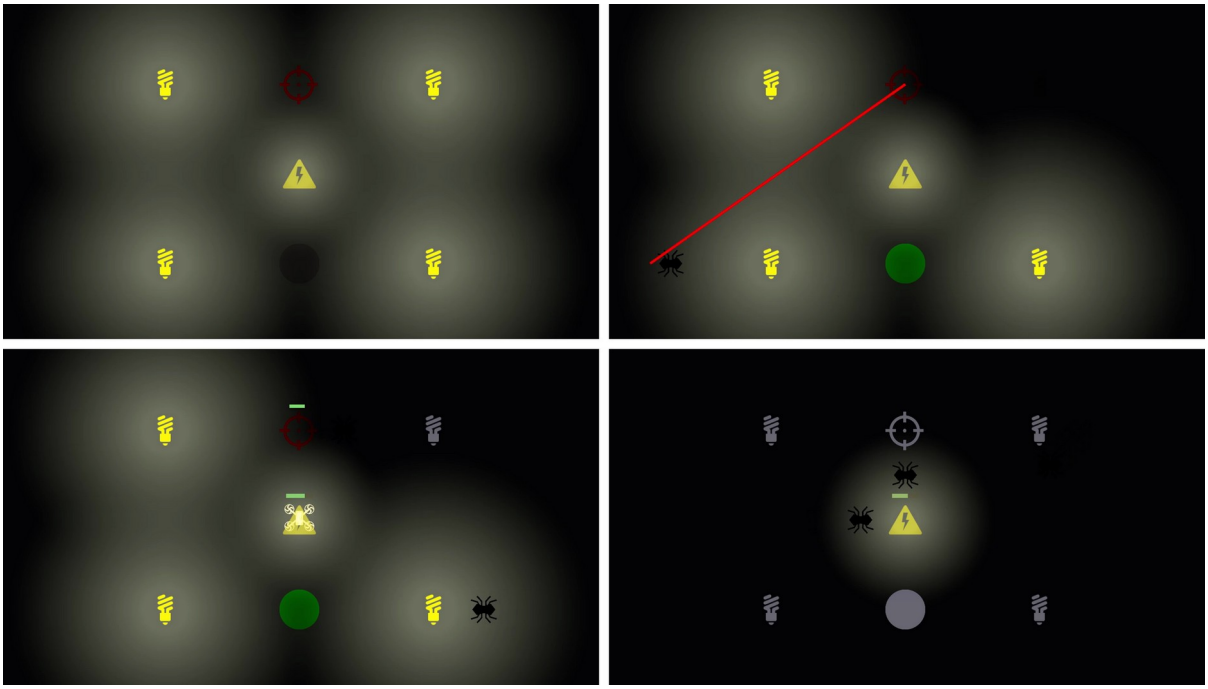
5. UI / UX

- **HUD** - power usage top-left (flashes red when a request is denied), scrap below it. Top-right: current phase, countdown, wave number and remaining enemies.
- **Health bars** - appear above a structure only once it is damaged, shifting from green to red.
- **Ruins** - show name and rebuild cost, but only during the preparation phase.

- **Pause menu** - Esc or P: Resume, Restart. Input on the map is blocked while paused.
- **Results screen** - victory or defeat, rank, statistics, restart button.
- **Controls** - left-click switches power and rebuilds ruins, Space starts the next wave, Esc pauses. No keyboard shortcuts per structure – the player looks at the map, not at a hotbar.

6. Artstyle

- All structure and enemy icons were designed by me: white symbols on black tiles, tinted per structure via the sprite color – warm yellow lamps, a red turret, a green drone station and a violet generator.
- Enemies have no tile, only dark silhouettes, so they disappear into the darkness and only become readable in the light.
- The look is carried by URP 2D lights: warm lamp light and a violet glow around the generator. Lasers, the flying drone and health bars use an unlit material so they glow even in complete darkness.
- The UI stays minimal so it doesn't compete with what the light reveals.

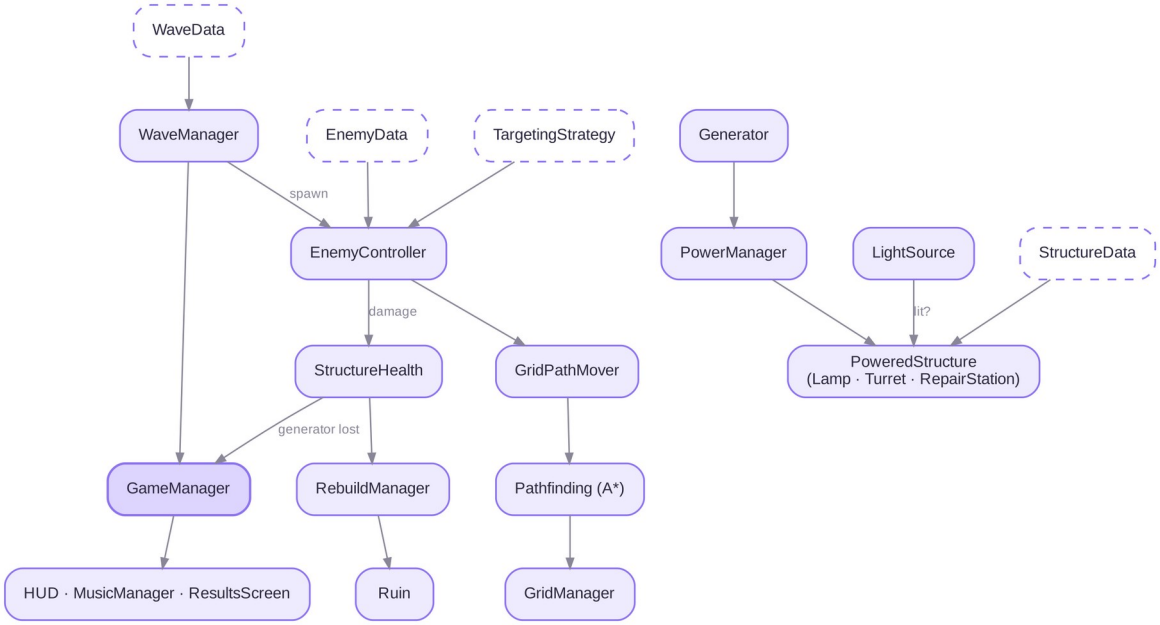


Preparation phase, turret firing at a lit Crawler, drone repairing during a wave, and the last lamp going out.

7. Technical Structure

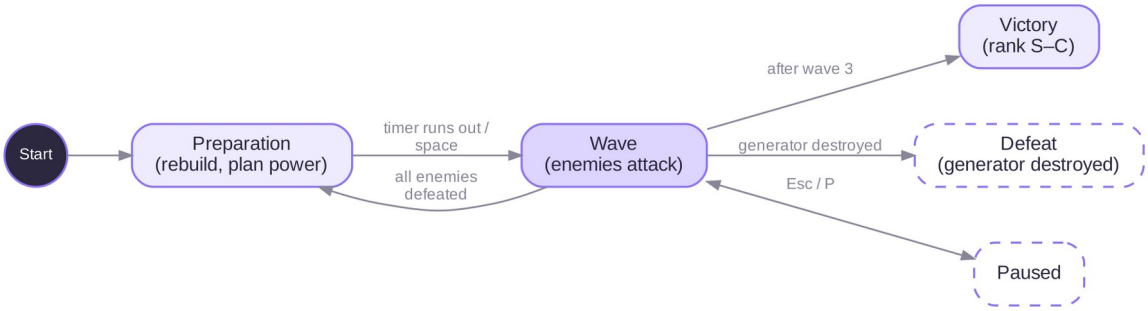
- **Grid & pathfinding** - a GridManager with 20 × 12 cells, tilemap obstacles and cell occupancy. A* with eight directions, no corner cutting and an octile heuristic. The GridPathMover re-plans as soon as its next cell gets blocked.
- **Enemy AI** - a state machine (Searching, Moving, Attacking, Dead). Target selection is swappable via TargetingStrategy ScriptableObjects.
- **Power system** - PowerManager with capacity, consumers and load shedding. Lamps, turret and repair station share the PoweredStructure base class and the IPowerConsumer interface.
- **Light as a query** - LightSource.IsPositionLit() answers whether a position is lit, using the same radius as the visible light.
- **Data-driven design** - StructureData, TurretData, RepairData, EnemyData, WaveData and TargetingStrategy as ScriptableObjects.
- **Interfaces** - IDamageable for anything that takes damage (structures, enemies), IPowerConsumer for anything that needs power.
- **Systems** - GameManager for phases, pause, restart and statistics, plus WaveManager, RebuildManager, MusicManager, HUD and ResultsScreen.

System Architecture



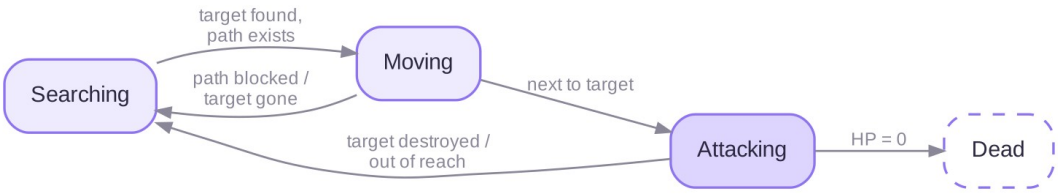
Dashed = data assets. Enemies, structures and waves get their values exclusively through ScriptableObjects.

Game State Flow



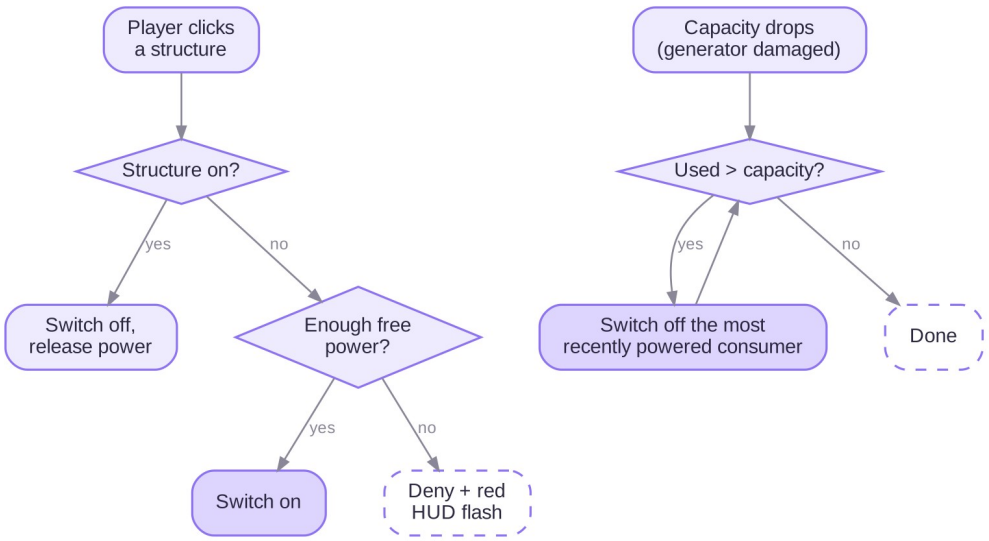
The GameManager drives the phases. Pause freezes the game via Time.timeScale = 0, while UI and music fades keep running on unscaled time.

Enemy State Machine



Crawler and Breaker use the same states – only their targeting strategy differs.

Power Flow & Load Shedding



Left: a click on a structure. Right: what happens when the capacity drops.

8. Scope

What's actually implemented:
1 map, 3 waves, 2 enemy types, 4 lamps, 1 turret, 1 repair drone, a generator, a power system with load shedding, A* pathfinding, ruins and rebuilding with scrap, a results screen with ranks, music with crossfades and stingers, a complete preparation/wave/pause/results flow, playable in the browser.

Deliberately out of scope:
Free placement of structures, multiple maps, upgrades, additional enemy types, progression between missions, sound effects.

Guiding principle: finishing and polishing within the defined scope takes priority over additional scope.

9. Possible Extensions

- **Scrap upgrades (planned)** - the next planned extension: spending scrap between waves on upgrades, for example larger light cones, a faster drone, quicker turret fire or stronger repairs. Scrap would then force a second decision: rebuild what was lost, or invest in getting stronger.
- **More content** - more waves, more enemy types and additional maps would be easy to add thanks to the data-driven setup. For the portfolio, the project is deliberately kept small: one mission that is finished and polished, rather than a bigger one that isn't.
- **Sound effects and animations** - feedback for switching power, laser hits and structures being destroyed.